Network Security — Special Project II

Department of Computer Science and Information Engineering

Known Plaintext Attack on a Custom Multi-Round Block Cipher

Cryptanalysis of Group 3's Poker–Permutation–Huffman Cipher

Subject Network Security
Topic Cipher Design and Cryptanalysis
Method Known Plaintext Attack (KPA)

May 28, 2026

Contents

1. Introduction

This report documents the cryptanalysis of a custom symmetric block cipher designed by Group 3 as part of the Network Security Special Project II assignment. The cipher processes four-character plaintexts drawn from a 36-character alphabet ({`0--9`, `a--z`}) and produces a binary ciphertext through two sequential encoding rounds.

The attack scenario is a **Known Plaintext Attack (KPA)**, the classical setting in which an analyst holds at least one plaintext–ciphertext pair and seeks to recover the secret key. Here, the pair is:

Item	Value
Known plaintext M_2	`book`
Corresponding ciphertext C_2	123-bit binary string
Target ciphertext C_1	140-bit binary string
Key space	base-36 strings of length 1 to 4

The objective is to recover the secret key from (M_2, C_2) and subsequently decrypt C_1 to obtain the unknown plaintext M_1.

2. Cipher Description

2.1 Alphabet and Key Encoding

The cipher operates over an alphabet of 36 characters:

$$\Sigma = \{\texttt{0, 1, 2, ..., 9, a, b, ..., z}\}$$

Each character is assigned a numeric value $v \in \{0, 1, \ldots, 35\}$ in lexicographic order. A key string K of length ℓ is converted to a single integer *seed master* (`sm`) via base-36 encoding:

$$\texttt{sm}(K) = \sum_{i=0}^{\ell-1} v(K_i) \cdot 36^{\,\ell-1-i}$$

2.2 Pseudorandom Number Generator

All deck shuffles and permutation tables are driven by a 32-bit Linear Congruential Generator (LCG):

$$s_{n+1} = (1664525 \cdot s_n + 1013904223) \bmod 2^{32}$$

This is the same parameterisation used in the Numerical Recipes family of LCGs and produces a full-period sequence over $[0, 2^{32})$.

2.3 Deck Construction — Fisher–Yates Shuffle

Given a seed value σ, a deck of 36 elements is produced by applying the Fisher–Yates (Knuth) shuffle from index 35 down to 0:

```
def make_deck(seed):
    deck = list(range(36))
    s = seed
    for i in range(35, -1, -1):
        s = lcg_next(s)
        j = s % (i + 1)
        deck[i], deck[j] = deck[j], deck[i]
    return deck
```

Listing 1: Deck construction via Fisher–Yates shuffle

2.4 Stage 1 — Poker Card Substitution

The poker substitution step replaces each input character using two pre-shuffled decks (`dA` and `dB`) in alternation. For a text of length n with input character values v_i:

$$\text{off}_i = (i + 1 + \text{prev}) \bmod 36, \qquad \text{idx}_i = (v_i + \text{off}_i) \bmod 36$$

$$r_i = \begin{cases} \texttt{dA}[\text{idx}_i] & \text{if } i \text{ is even} \\ \texttt{dB}[\text{idx}_i] & \text{if } i \text{ is odd} \end{cases} \qquad \text{prev} \leftarrow r_i$$

The initial value of `prev` is `sm` mod 36. Deck seeds are XOR-derived from `sm` using constants specific to each round (detailed in Section 3).

2.5 Stage 2 — Permutation

A position permutation π of length n is generated by drawing n values from the LCG, then sorting their indices by value (ascending). The text is reordered accordingly:

$$T'_i = T_{\pi(i)}, \quad i = 0, 1, \ldots, n-1$$

The permutation seed is XOR-derived from `sm` with a round-specific constant.

2.6 Stage 3 — Score-Tree (Huffman) Encoding

A Huffman tree is constructed for the full 36-character alphabet using *scores* computed from a seed η:

$$\text{score}_i = \big((i + 1) \cdot \eta + 17\big) \bmod 997, \quad i = 0, 1, \ldots, 35$$

Characters with lower scores receive longer codes (they are treated as less frequent). The heap ordering uses `(score, counter, node)` to break ties deterministically by insertion order.

After encoding each character, the seed is updated via SHA-256:

$$\eta' = \texttt{SHA-256}(\,\texttt{str}(\eta \oplus \texttt{ord}(c))\,)$$

where c is the character just encoded and $\texttt{ord}(c)$ is its ASCII value.

The initial score seed for each round is:

$$\eta_{\mathrm{R}n} = \sum_{c \in K + \text{“R}n\text{”}} \texttt{ord}(c)$$

3. Two-Round Structure

The cipher applies the three-stage pipeline twice. All XOR constants follow a uniform scaling rule: **Round** n **uses** $n \times$ `0x1111`, $n \times$ `0x2222`, **and** $n \times$ `0x3333` as its XOR offsets for deck A, deck B, and the permutation, respectively.

Table 1: XOR constants and score seeds for each round

Round	Poker Deck A	Poker Deck B	Permutation	Score Seed
Round 1	`sm` $\oplus$ `0x1111`	`sm` $\oplus$ `0x2222`	`sm` $\oplus$ `0x3333`	η_{R1}
Round 2	`sm` $\oplus$ `0x2222`	`sm` $\oplus$ `0x4444`	`sm` $\oplus$ `0x6666`	η_{R2}

The full encryption pipeline is:

$$M \xrightarrow{\text{Poker}_1} S_1 \xrightarrow{\text{Perm}_1} T_1 \xrightarrow{\text{Huffman}_1} R_1 \xrightarrow{\text{Poker}_2} S_2 \xrightarrow{\text{Perm}_2} T_2 \xrightarrow{\text{Huffman}_2} C$$

Round 1 converts the four-character plaintext into a binary string R_1. Round 2 treats each bit of R_1 as a character (values 0 or 1 in Σ), applies poker substitution to map them into the full 36-character alphabet, permutes the result, and encodes via a second Huffman tree to produce the final binary ciphertext.

4. Known Plaintext Attack — Methodology

4.1 Key Space

The secret key is a string of length 1 to 4 over Σ, giving a total key space of:

$$|\mathcal{K}| = 36^1 + 36^2 + 36^3 + 36^4 = 36 + 1{,}296 + 46{,}656 + 1{,}679{,}616 = 1{,}727{,}604$$

This size is small enough for exhaustive search.

4.2 Attack Strategy

A direct brute-force approach is impractical at scale when each candidate key requires multiple SHA-256 calls and Huffman tree constructions per character. The attack was therefore split into two phases.

Phase 1 — Reverse Decoding of C_2

Rather than encoding forward for every key, the ciphertext C_2 is decoded *backwards* under all plausible initial seeds and seed-update formulas to obtain the set of all strings that could be the input T_2 to the final Huffman step. Specifically, for each seed $\eta \in \{0, 1, \ldots, 996\}$ (the effective range modulo 997) and each of seven seed-update formulas, the decoder reads C_2 bit by bit and attempts to recover exactly L characters (for $L \in \{20, 21, \ldots, 30\}$), consuming all 123 bits with no remainder.

This pre-computation yields a set $\mathcal{T}$ of 662 candidate strings grouped by length, computed once and held in memory.

Phase 2 — Forward Key Search with String Matching

For each candidate key K:

1. Compute $\texttt{sm}(K)$ and execute Round 1 on $M_2 = \texttt{book}$, obtaining R_1.
2. For each combination of XOR constants $(\texttt{xA}, \texttt{xB}) \in \{k \cdot \texttt{0x1111}\}^2$ and permutation constant $\texttt{pc} \in \{k \cdot \texttt{0x1111}\}$:
   - Apply poker substitution with `xA`, `xB` to R_1.
   - Apply permutation with seed $\texttt{sm} \oplus \texttt{pc}$.
   - Check whether the resulting string belongs to $\mathcal{T}$.
3. On a match, verify by running the full encryption and comparing against C_2 exactly.

This approach replaces repeated SHA-256 and Huffman computations with a simple set-membership test, substantially reducing per-key cost.

4.3 Parallelisation

The key search was distributed across all available CPU cores using Python's `multiprocessing.Pool`. Keys were divided into chunks of 500 and dispatched to worker processes that share the pre-computed $\mathcal{T}$ via an initialiser. The pool terminates as soon as any worker returns a valid key.

5. Results

5.1 Key Recovery

The brute-force search identified the following key:

Parameter	Value
Secret key K	`4z1j`
Seed master `sm`	232,039
Score seed η_{R1}	460
Score seed η_{R2}	461
Round 2 poker constants	`sm` $\oplus$ `0x2222`, `sm` $\oplus$ `0x4444`
Round 2 perm constant	`sm` $\oplus$ `0x6666`

5.2 Decryption of C_1

Using key `4z1j`, the decryption of C_1 proceeds by reversing both rounds in sequence:

1. Decode C_1 (140 bits) via the Round 2 Huffman tree (seed $\eta_{R2} = 461$) to obtain T_2 (24 characters).
2. Apply the inverse permutation (seed `sm` $\oplus$ `0x6666`).
3. Apply inverse poker substitution (constants `0x2222`, `0x4444`) to recover R_1 (binary, 24 bits).
4. Decode R_1 via the Round 1 Huffman tree (seed $\eta_{R1} = 460$) to obtain 4 characters.
5. Apply the inverse permutation (seed `sm` $\oplus$ `0x3333`).
6. Apply inverse poker substitution (constants `0x1111`, `0x2222`) to recover M_1.

$$M_1 = \texttt{heir}$$

5.3 Intermediate Values — $M_2 =$ `book`

Table 2: Step-by-step encryption trace for $M_2 =$ `book`

Step	Operation	Input	Output
R1.1	Poker sub (`0x1111/0x2222`)	`book`	`1410`
R1.2	Permutation (`sm`$\oplus$`0x3333`)	`1410`	`0114`
R1.3	Huffman encode (seed 460)	`0114`	`0111110101011001010100` (22 bits)
R2.1	Poker sub (`0x2222/0x4444`)	22-bit R_1	`nkaaim6arf78plb0lxil9q`
R2.2	Permutation (`sm`$\oplus$`0x6666`)	above	`0nmfaklalr7ix96iaqbl8p`
R2.3	Huffman encode (seed 461)	above	C_2 (123 bits)

5.4 Intermediate Values — $M_1 =$ `heir`

Table 3: Step-by-step encryption trace for $M_1 =$ `heir`

Step	Operation	Input	Output
R1.1	Poker sub (`0x1111/0x2222`)	`heir`	`3rqm`
R1.2	Permutation (`sm`$\oplus$`0x3333`)	`3rqm`	`mq3r`
R1.3	Huffman encode (seed 460)	`mq3r`	`101011011000001101100101` (24 bits)
R2.1	Poker sub (`0x2222/0x4444`)	24-bit R_1	`my3bl0fyo3sx1a7yjfg40iel`
R2.2	Permutation (`sm`$\oplus$`0x6666`)	above	`ym03byjy4eosllf0fg3i7ax1`
R2.3	Huffman encode (seed 461)	above	C_1 (140 bits)

5.5 Verification

Both results were verified by running the forward encryption and comparing the output against the original ciphertexts bit by bit:

Test	Expected	Result
`encrypt("book", "4z1j")` $= C_2$	123 bits	**Match**
`encrypt("heir", "4z1j")` $= C_1$	140 bits	**Match**

6. Discussion

6.1 Why the Cipher is Vulnerable to KPA

The cipher's susceptibility to an exhaustive key search under KPA conditions stems from two factors. First, the key space of roughly 1.7 million candidates is well within the reach of a modern multi-core processor in under two minutes, with no specialised hardware required. Second, the deterministic nature of all three stages — the LCG shuffle, the positional poker substitution, and the score-seeded Huffman tree — means that for a given key, the entire encryption is reproducible exactly. There is no randomisation (nonce, IV, or salt) introduced per message, so a single known pair is sufficient to identify the key by exhaustive search.

6.2 The XOR Constant Pattern

An interesting structural property of the cipher is the uniform scaling of XOR constants across rounds. Round n derives all three of its seeds (deck A, deck B, permutation) by multiplying the base triplet (`0x1111`, `0x2222`, `0x3333`) by n. This regularity is consistent with the slide 12 formula for permutation seeds (pid $\times$ `0x3333`, where pid is the round index) and extends naturally to the poker step. While elegant as a design principle, it means that an analyst who has recovered Round 1's constants can immediately infer Round 2's constants without any additional information.

6.3 Strengthening Recommendations

Several straightforward measures would significantly raise the cost of a KPA against this cipher:

- **Longer keys.** Extending the key to 8 characters raises the search space to $36^8 \approx 2.8 \times 10^{12}$, placing exhaustive search out of reach without purpose-built hardware.
- **Message-specific randomisation.** Prepending a random nonce to each message and incorporating it into the seed derivation would invalidate any pre-computed table derived from a single known pair.
- **Independent round constants.** Deriving round constants from a key-dependent function rather than a fixed arithmetic pattern would prevent an analyst from inferring Round 2 parameters from Round 1.
- **Replacing the LCG.** The 32-bit LCG has a period of 2^{32} and is statistically weak; replacing it with a cryptographic PRNG (e.g. ChaCha20 or AES-CTR) would remove the structural predictability in the deck construction.

7. Conclusion

This report presented a complete Known Plaintext Attack on Group 3's custom poker–permutation–Huffman cipher. Starting from the single known pair $(M_2, C_2) = (\texttt{book}, C_2)$, the secret key `4z1j` was recovered by exhaustive search over a key space of 1,727,604 candidates. The attack combined a backwards decoding phase — which pre-computed all valid intermediate strings from C_2 without knowing the key — with a fast string-matching forward search that avoided repeated Huffman computations.

With the recovered key, the previously unknown plaintext of C_1 was determined to be:

$$\boxed{M_1 = \texttt{heir}}$$

Both results were confirmed by re-encrypting each plaintext and verifying an exact bit-for-bit match against the original ciphertexts.

Appendix — Source Files

Three Python scripts implement the full attack and are available in the project repository:

File	Purpose
`crack_v10.py`	KPA brute force — recovers key and decrypts M_1
`decrypt_c1.py`	Step-by-step decryption trace for C_1
`final_verify.py`	Bit-for-bit forward encryption verification

All scripts require only Python 3 standard library modules (`hashlib`, `heapq`, `multiprocessing`) and produce identical output on any platform. No third-party packages are needed.

Repository: https://github.com/RazaHassan491/-2-3-.....git